Modern Computing Systems Design (CSE 2221)

3rd SEMESTER – CSE (AIML, DS, IOT, CYSC)

Chapter-3 : Processor Element

- This chapter introduces the conceptual design and core functionality of a modern processor, focusing on the internal mechanisms that allow it to execute instructions, manage data, and communicate with the outside world.

Chapter-3 : Processor Element

The following topics will be covered in this chapter:

- A simple processor
- The instruction set
- Addressing modes
- Instruction categories
- Interrupt processing
- Input/output operations

Chapter-3 : Processor Element

- A Simple Processor

Central Processing Unit (CPU)

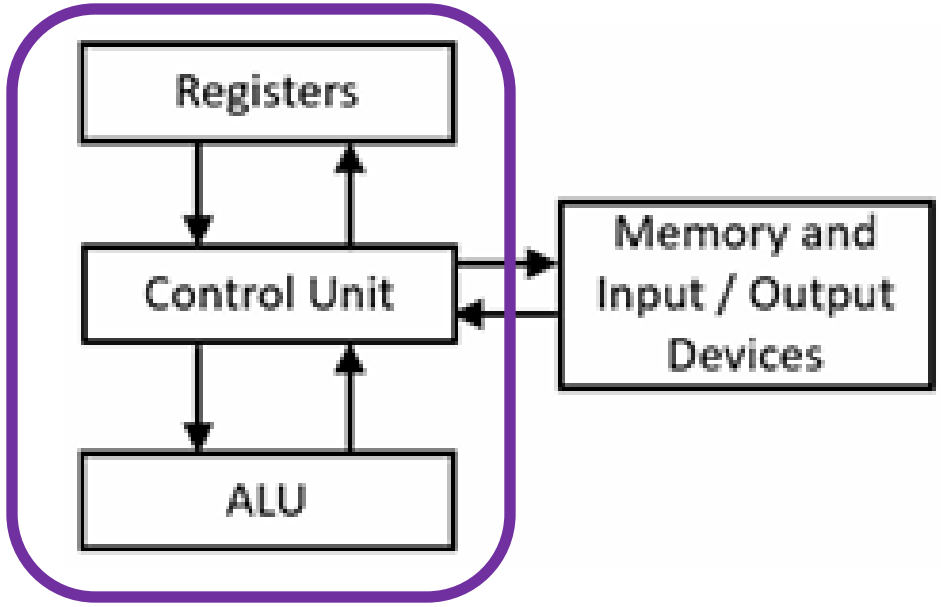


Figure 3.1: Interactions between processor functional units

Chapter-3 : Processor Element

- A Simple Processor
- The fundamental architectural units common to almost all CPUs:
  - Control Unit :
    - Acts as the brain of the processor.
    - Fetches instructions from memory.
    - Decodes the instructions to determine the required operation.
    - Generates control signals to coordinate the ALU, registers, and other components to execute the instruction.
    - Manages the sequence of execution.

Chapter-3 : Processor Element

❑ Control Unit :

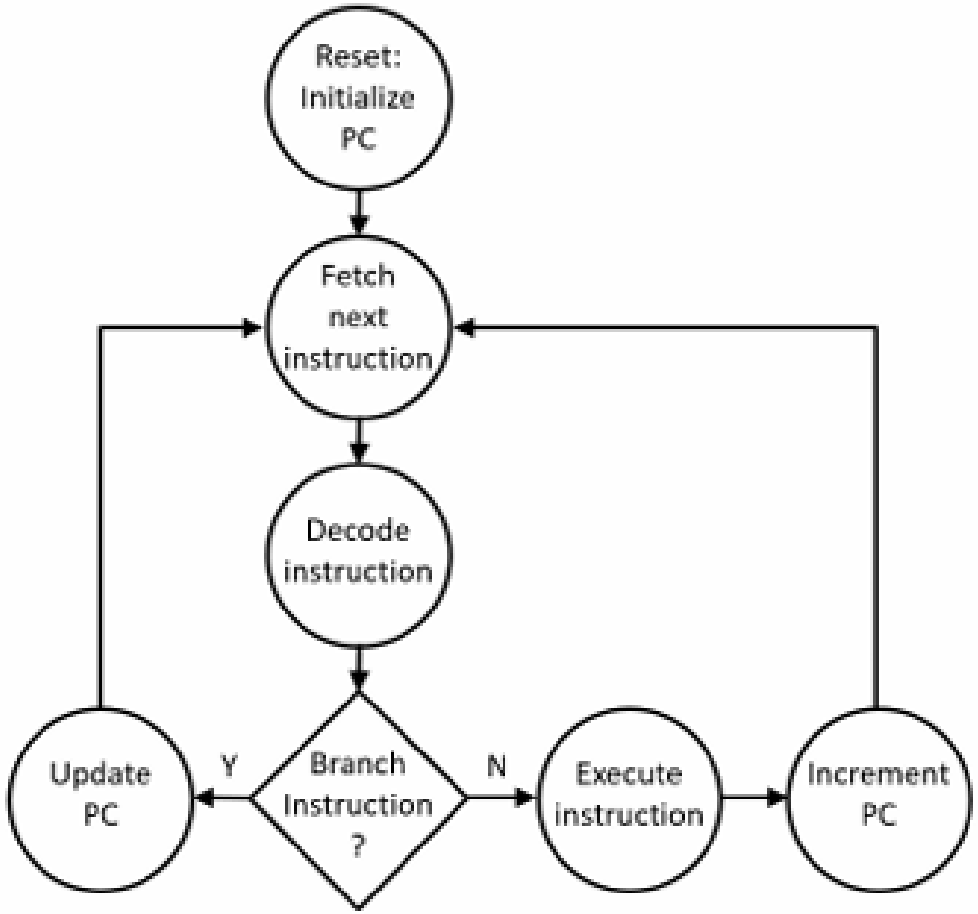


Figure 3.2: Instruction execution cycle

Chapter-3 : Processor Element

❑ Control Unit :

❖ Executing an Instruction:

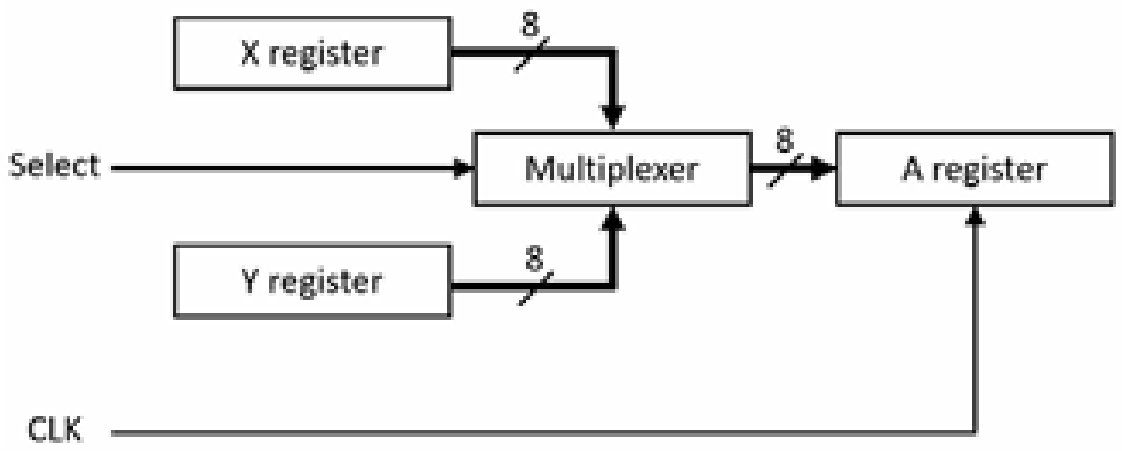


Figure 3.3: 6502 TXA and TYA instructions

Chapter-3 : Processor Element

- Arithmetic Logic Unit (ALU) :
  - The execution unit for all computational tasks.
  - Performs Arithmetic Operations (addition, subtraction, multiplication, division).
  - Performs Logical Operations (AND, OR, NOT, XOR).

Chapter-3 : Processor Element

❑ ALU Unit :

Example: ADC and SBC instructions

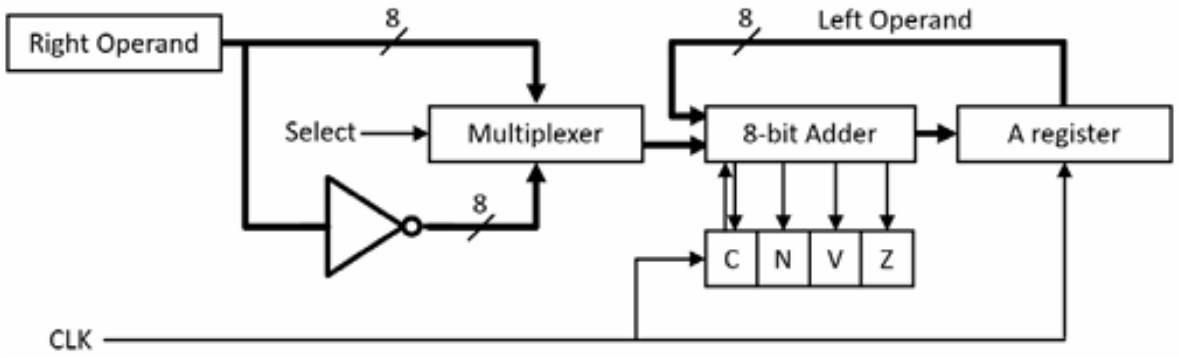


Figure 3.4: 6502 addition and subtraction operations

Chapter-3 : Processor Element

- Registers:
  - Small, ultra-fast storage locations located inside the CPU.
  - They hold data, addresses, and status information actively being used by the current instruction.
  - Types include general-purpose registers (for data manipulation), program counter (holds the address of the next instruction), and status/flag registers (reflect results of operations).

Chapter-3 : Processor Element

- **Registers:**
  - **RISC Architecture vs CISC Architecture :**

RISC	CISC
It stands for Reduced Instruction Set Computer.	It stands for Complex Instruction Set Computer.
It is a microprocessor architecture that uses small instruction set of uniform length.	This offers hundreds of instructions of different sizes to the users.
These simple instructions are executed in one clock cycle.	This architecture has a set of special purpose circuits which help execute the instructions at a high speed.
These chips are relatively simple to design.	These chips are complex to design.
They are inexpensive.	They are relatively expensive.
More numbers of registers	Less number of registers

Chapter-3 : Processor Element

- **Registers:**
  - **RISC Architecture vs CISC Architecture :**

RISC	CISC
Most of the operations are based on registers which reduces the need to access memory	Most of the operations are memory based
It has less number of instructions.	It has more number of instructions.
It has fixed-length encodings for instructions.	It has variable-length encodings of instructions.
Simple addressing formats are supported.	The instructions interact with memory using complex addressing modes.
Separate LOAD/Store instructions are used to access memory operand.	No Separate LOAD/STORE instructions are available
Examples of RISC chips include ARM, RISC-V	Examples of CISC include x86 Architecture

Chapter-3 : Processor Element

❑ Addressing Modes:

- The way in which an operand is denoted in an instruction is known as addressing modes.

- Addressing modes are techniques used by the CPU to identify the location of the operand(s) needed for executing an instruction. They provide rules for interpreting the address field in an instruction, helping the CPU fetch operands correctly.

Opcode – Tells the CPU what operation to perform (e.g., ADD, MOV).

Operands – The data or addresses on which the operation is performed.

Chapter-3 : Processor Element

❑ Addressing Modes:

There are 9 different addressing modes used in 6502 processor.

1. Immediate Addressing mode
2. Absolute Addressing mode
3. Absolute indexed addressing mode
4. Indirect indexed addressing mode
5. Indexed indirect addressing mode
6. Absolute indirect addressing mode
7. Zero-page addressing mode
8. Implicit (Implied) addressing mode
9. Accumulator addressing mode

Chapter-3 : Processor Element

❑ Addressing Modes:

1. Immediate addressing mode: In this addressing mode the value (data operand) is provided in the instruction itself.

Ex: LDA #$123 (the value after # indicates the data)

ADC #12

2. Absolute addressing mode: The addressing mode in which the 16-bit address of the memory is provided in the instruction.

Ex: CMP $FE02 ; compare A with byte at &FE02

3. Absolute indexed addressing mode: The addressing mode in which the memory address is computed by adding a base address provided in the instruction to a value contained in the X or Y register.

Ex: LDA $123, X ($123 specifies the address) -> Absolute X-Indexed addressing mode

ADC $432, Y ($123 specifies the address) -> Absolute Y-Indexed addressing mode

4. Indirect indexed addressing mode: In this addressing mode, the memory location/address of the operand is calculated by adding the content of Y register with the content of base address which is 8-bit address.

Ex: ADC ($10), Y ($10 specifies the address)

Chapter-3 : Processor Element

❑ Addressing Modes:

5. Zero page addressing mode:

When a zero page address (0x00-0xFF, refers to the high byte of 16-bit address) is used, it is called as **Zero page addressing mode**.

Ex: LDA $C5

6. Indexed indirect addressing mode: In this addressing mode, the memory location/address of the operand is calculated by adding the content of X register with base address which is 8-bit address.

Ex: LDA ($10, X)

6. Absolute indirect addressing mode: The addressing mode that uses 16-bit pointer to final address.

Ex: JMP ($1234)

7. Implicit (Implied) addressing mode: In this addressing mode, no operand is specified in the instruction (instruction implies the data itself)

Ex: PHA

RST

Addressing Modes:

8. Relative addressing mode: In this addressing mode Branch target as signed offset from PC

Ex: BEQ $34

BNE $34

9. Accumulator addressing mode: The addressing mode in which the operation is performed with/to the accumulator.

Ex: ASL,

LSR,

Chapter-3 : Processor Element

Instruction Set of 6502 processor:

A functional breakdown of the 6502-instruction set is commonly categorized as follows:

- **Load/store instructions:**

  These instructions move data between the processor's registers and memory.

  - LDA (Load Accumulator)
  - LDX (Load X Register)
  - LDY (Load Y Register)
  - STA (Store Accumulator)
  - STX (Store X Register)
  - STY (Store Y Register)

- **Register transfer instructions:**

  These instructions move data between the processor's main registers.

  - TAX (Transfer Accumulator to X)
  - TAY (Transfer Accumulator to Y)
  - TXA (Transfer X to Accumulator)
  - TYA (Transfer Y to Accumulator)

Chapter-3 : Processor Element

Instruction Set of 6502 processor:

Stack instructions:

These instructions handle pushing and pulling data to and from the stack, which resides on memory page $01.

- PHA (Push Accumulator)
- PHP (Push Processor Status)
- PLA (Pull Accumulator)
- PLP (Pull Processor Status)
- TSX (Transfer Stack Pointer to X)
- TXS (Transfer X to Stack Pointer)

Arithmetic and logical instructions:

This category includes instructions for mathematical operations and logical bit manipulation.

- ADC (Add with Carry)
- SBC (Subtract with Carry)
- AND (Logical AND)
- EOR (Exclusive OR)
- ORA (Logical OR)
- ASL (Arithmetic Shift Left)
- LSR (Logical Shift Right)
- ROL (Rotate Left)
- ROR (Rotate Right)
- INC (Increment Memory)
- INX (Increment X Register)
- INY (Increment Y Register)
- DEC (Decrement Memory)
- DEX (Decrement X Register)
- DEY (Decrement Y Register)

Chapter-3 : Processor Element

❑ Instruction Set of 6502 processor:

- **Comparison and bit test instructions:**
  These instructions compare values and set the status register flags accordingly.CMP (Compare Accumulator).

  - CPX (Compare X Register)
  - CPY (Compare Y Register)
  - BIT (Bit Test)

- **Conditional branch instructions:**
  These instructions alter the program flow based on the state of the status flags.

  - BCC (Branch if Carry Clear)
  - BCS (Branch if Carry Set)
  - BEQ (Branch if Equal)
  - BMI (Branch if Minus)
  - BNE (Branch if Not Equal)
  - BPL (Branch if Plus)
  - BVC (Branch if Overflow Clear)
  - BVS (Branch if Overflow Set)

Chapter-3 : Processor Element

Instruction Set of 6502 processor:

- **Unconditional jump instructions:**

  These instructions change the program counter to a new address.

  - JMP (Jump)
  - JSR (Jump to Subroutine)
  - RTS (Return from Subroutine)
  - RTI (Return from Interrupt)

- **Flag control instructions:**

  These instructions are used to manually set or clear specific flags in the status register.

  - CLC (Clear Carry Flag)
  - CLD (Clear Decimal Flag)
  - CLI (Clear Interrupt Disable)
  - CLV (Clear Overflow Flag)
  - SEC (Set Carry Flag)
  - SED (Set Decimal Flag)
  - SEI (Set Interrupt Disable)

- **System instructions:**

  This category covers miscellaneous system-level functions.

  - BRK (Force Interrupt)
  - NOP (No Operation)
  - SEI
  - CLI

Chapter-3 : Processor Element

- **Input/Output (I/O) Operations:**
  - The goal of the I/O portion of a processor architecture is to efficiently transfer data between external peripheral devices and system memory.
  - The processor uses the instruction categories, addressing modes, and interrupt processing methods for interactions with I/O devices.
  - Two main approaches employed in modern processors to access I/O devices.

    - Memory-Mapped I/O:
      - where I/O registers are accessed like memory locations, used by 6502, ARM
      - It dedicates portions of the system address space to I/O devices. The processor accesses peripheral devices at predefined addresses using the same instructions and addressing modes it uses to read and write system memory.

    - Port-Mapped I/O :
      - where a separate address space and special I/O instructions are used, like in x86 architecture).
      - I/O devices are assigned port numbers as addresses.

Chapter-3 : Processor Element

- **Input/Output (I/O) Operations:**

- Programmed I/O (Polling):The CPU constantly checks the status register of the I/O device in a loop, waiting for it to be ready. Simple, but highly inefficient due to wasted CPU cycles.

- Interrupt-Driven I/O: The CPU initiates a command, then moves on to other tasks. The I/O device generates an interrupt signal when the operation is complete or when it needs service, making the process much more efficient.

- Direct Memory Access (DMA):A separate DMA controller handles the direct transfer of large blocks of data between memory and an I/O device (e.g., a disk drive).The CPU only initiates the transfer and is only interrupted when the entire block transfer is complete, maximizing CPU availability.

Chapter-3 : Processor Element

- **Interrupt Processing** : Describe how the processor handles asynchronous external events and high-priority tasks:

- Interrupts: Signals that temporarily halt the main program execution to run a specialized routine (Interrupt Service Routine or ISR).

- $\overline{IRQ}$ (Interrupt Request) Processing: Active low signal/pin. Handled by a signal line; these are typically maskable (can be ignored by the processor if disabled). This method implements the hardware IRQ interrupt. It checks if the Interrupt disable flag is set, and if not, it performs the interrupt sequence.

- NMI (Non-Maskable Interrupt) Processing: The highest priority interrupt, often used for critical system failures (like a power-loss warning), and cannot be ignored. It cannot be disabled by the Interrupt flag.

- BRK Instruction Processing: A software interrupt initiated by an instruction within the code, typically used to trigger a breakpoint during debugging

Chapter-3 : Processor Element

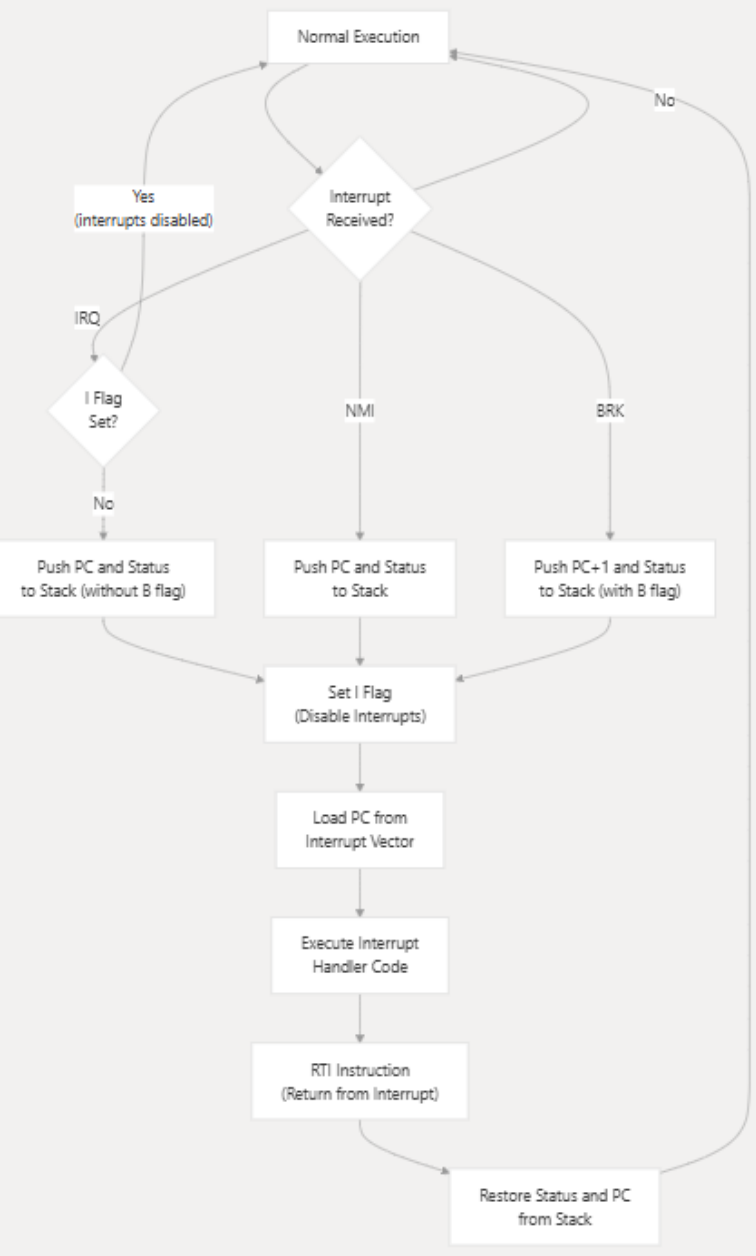
Interrupt Processing : MOS6502 processor supports three types of interrupts, each with its own vector location and handling behavior:

nterrupt Type	Vector Location	Maskable	Description
IRQ	oxFFFE-oxFFFF	Yes	Hardware interrupt that can be disabled by setting the I flag
NMI	oxFFFA-oxFFFB	No	Non-maskable interrupt that always triggers regardless of I flag
BRK	oxFFFE-oxFFFF	Yes	Software interrupt triggered by the BRK instruction
Reset	oxFFFC-oxFFFD	No	System reset (not technically an interrupt)

Chapter-3 : Processor Element

Interrupt Handling Process:

:



Chapter-3 : Processor Element

Interrupt Processing :

Detailed Interrupt Sequence

The following steps show the exact sequence of operations performed during interrupt handling:

IRQ (Interrupt Request)

1. Check if the Interrupt disable (I) flag is clear
2. If I is set, the interrupt is ignored
3. Push the high byte of PC to the stack (decrements SP)
4. Push the low byte of PC to the stack (decrements SP)
5. Push the status register to the stack with B flag clear (decrements SP)
6. Set the I flag to disable further interrupts
7. Load the new PC from the IRQ vector (0xFFFE-0xFFFF)
8. Continue execution at the new PC

Chapter-3 : Processor Element

Interrupt Processing :

Detailed Interrupt Sequence

The following steps show the exact sequence of operations performed during interrupt handling:

NMI (Non-Maskable Interrupt)

1. Push the high byte of PC to the stack (decrements SP)
2. Push the low byte of PC to the stack (decrements SP)
3. Push the status register to the stack with B flag clear (decrements SP)
4. Set the I flag to disable further interrupts
5. Load the new PC from the NMI vector (0xFFFA-0xFFFB)
6. Continue execution at the new PC

Chapter-3 : Processor Element

Interrupt Processing :

Detailed Interrupt Sequence

The following steps show the exact sequence of operations performed during interrupt handling:

BRK (Software Interrupt)

BRK is an instruction (opcode 0x00) rather than a hardware signal. When executed:

1. Increment PC (to skip the byte following the BRK opcode)
2. Push the high byte of PC to the stack (decrements SP)
3. Push the low byte of PC to the stack (decrements SP)
4. Push the status register to the stack with B flag set (decrements SP)
5. Set the I flag to disable further interrupts
6. Load the new PC from the IRQ/BRK vector (0xFFFE-0xFFFF)
7. Continue execution at the new PC

RTI (Return from Interrupt)

The RTI instruction (opcode 0x40) restores the processor state after an interrupt:

1. Pull the status register from the stack (increments SP)
2. Pull the low byte of PC from the stack (increments SP)
3. Pull the high byte of PC from the stack (increments SP)
4. Continue execution at the restored PC